

实验九 Linux 系统 fork 等系统调用实验

9.1 使用 ps 命令查看进程信息

【实验内容】

ps 是基本的 Linux 命令，通过本实验，不仅要熟悉 ps 命令方法，更重要的是可以了解 Linux 进程的组成。

【实验原理】

ps: 查看系统中的进程，Linux 中可以使用 ps -aux 查看所有进程。其中 PID 代表进程 ID，TTY 是该进程是由哪个控制台启动的，CMD 则是命令。

如果想列出更详细的信息，则可使用命令：“ps -auxw”。参数 w 表示加宽显示的命令行，参数 w 可以写多次，通常最多写 3 次，表示加宽 3 次，这足以显示很长的命令行了。

【实验内容】

在 shell 提示符下输入如下命令，并解释输出的结果：

```
[root@vm root]#ps
```

```
PID TTY TIME CMD
```

```
16767 pts/1 0:00 ps
```

```
18029 pts/1 0:00 bash
```

```
[root@vm root]#ps aux
```

```
PID TTY TIME CMD
```

```
...
```

9.2 使用 proc 文件系统查看进程信息

【实验目的】

本实验将指导学员了解 proc 文件系统，通过 proc 文件系统查询进程信息，可以扩展到修改系统参数。

【实验原理】

/proc 文件系统是一个虚拟文件系统，通过文件系统接口实现对内核的访问，输出系统运行状态。它以文件系统的形式，为操作系统本身和应用进程之间的通信提供了一个界面，使应用程序能够安全，方便的获得系统当前的运行状况和内核的内部数据信息，并且可以修改某些系统的配置信息。

【实验内容】

1)认识 proc 文件系统的文件和目录

```
[root@vm root]#cd /proc
```

```
[root@vm root]#ls
```

2)通过 proc 文件系统查看系统当前进行状态

```
[root@vm root]#cat /proc/self/status
```

3)查询文件句柄的当前使用情况

```
# cat /proc/sys/fs/file-nr
```

```
426      15252458
```

file-nr 文件显示了三个参数：分配的文件句柄总数、当前使用的文件句柄数以及可以分配的最大文件句柄数。如果需要增大 /proc/sys/fs/file-max 中的值，请确保正确设置 ulimit。对于 2.4.20，通常将其设置为 unlimited。使用 ulimit 命令来验证 ulimit 设置：

```
[root@vm root]# ulimit
```

```
unlimited
```

4)通过 proc 文件系统修改内核中预定的一些变量

1) 修改整个系统中文件句柄的最大数量

```
[root@vm root]#ls /proc/sys/fs/file-max
```

```
52458
```

```
[root@vm root]#echo 65536 >/proc/sys/fs/file-max
```

```
[root@vm root]#ls /proc/sys/fs/file-max
```

```
65536
```

2) 修改网络 TTL

```
[root@vm root]#ls /proc/sys/net/ipv4/ip_default_ttl
```

```
64
```

```
[root@vm root]#echo 128 >/proc/sys/net/ipv4/ip_default_ttl
```

```
[root@vm root]#ls /proc/sys/net/ipv4/ip_default_ttl
```

```
128
```

3) 修改系统中最大进程数量

```
[root@vm root]#ls /proc/sys/kernel/pid_max
```

```
32768
```

```
[root@vm root]#echo 65536 >/proc/sys/kernel/pid_max
```

```
[root@vm root]#ls /proc/sys/kernel/pid_max
```

```
65536
```

4) 修改普通用户的最大 RTC 频率

```
[root@vm root]#ls /proc/sys/dev/rtc/max-user-freq
```

```
64
```

```
[root@vm root]#echo 128 >/proc/sys/dev/rtc/max-user-freq
```

```
[root@vm root]#ls /proc/sys/dev/rtc/max-user-freq
```

```
128
```

5) 其他一些信息

```
[root@vm root]#cat /proc/cpuinfo - CPU
```

```
[root@vm root]#cat /proc/interrupts - 中断
```

```
[root@vm root]#cat /proc/ioports - 设备 IO 端口
[root@vm root]#cat /proc/meminfo - 内存信息(i.e. mem used, free, swap size)
[root@vm root]#cat /proc/partitions - 所有设备的所有分区
[root@vm root]#cat /proc/pci - PCI 设备的信息
[root@vm root]#cat /proc/swaps - 所有 Swap 分区的信息
[root@vm root]#cat /proc/version - Linux 的版本号
```

9.3 使用 fork、exit 和 exec 系统调用编写多进程程序

【实验目的】

本实验将通过编写 fork 等系统调用的程序，加深对系统进程及其控制的了解。

【实验原理】

fork 后父子进程会同步运行，但父子进程的返回顺序是不确定的。设两个变量 global 和 test 来检测父子进程共享资源的情况。同时在进程退出时对 exit 和 _exit 的区别进行测试和说明。

【实验内容】

1. fork

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>
int global=22;
char buf[]="the test content!\n";

int main(void)
{
    int test=0,stat;
    pid_t pid;
    if(write(STDOUT_FILENO, buf, sizeof(buf)) != sizeof(buf))
        { perror("write error!"); }
    printf(" fork test!\n");
    /* fork */
    pid = fork();    /*we should check the error*/
    if (pid == -1)
    {
        perror("fork");
        exit(0);
    }
    else if (pid == 0)
```

```
{
    global++;    test++;
    printf("global=%d test=%d Child,my PID is %d\n",global,test,getpid());
    exit(0);
}
/*else be the parent*/
    global+=2;
    test+=2;
    printf("global=%d test=%d Parent,my PID is %d\n",global,test,getpid());
    exit(0);
//printf("global=%d test=%d Parent,my PID is %d",global,test,getpid());
//_exit(0);
}
```

编译执行，并分析结果：

```
[root@localhost root]# ./test
the test content!
fork test!
global=23 test=1 Child,my PID is 2751
global=24 test=2 Parent,my PID is 2750
```

可以看出父子进程打印出了各自的进程号和对应变量的值，显然 `global` 和 `test` 在父子进程间是独立的，其各自的操作不会对对方的值有影响。将上述代码最后的两行代码替换为注释掉的 `_exit(0)` 行，重新编译，查看结果，解释原因：

```
[root@localhost root]# ./test
the test content!
fork test!
global=23 test=1 Child,my PID is 2771
```

父进程的信息没有打印出来，其原因是：`_exit()` 函数直接使进程停止运行，清除其使用的内存空间，并销毁其在内核中的各种数据结构；而 `exit()` 函数则在这些基础上作了一些包装，在执行退出之前加了若干道工序。`exit()` 函数在调用 `exit` 系统调用之前要检查文件的打开情况，把文件缓冲区中的内容写回文件，即会“清理 I/O 缓冲”。若将上述 `_exit(0)` 改为 `exit(0)`，则肯定会有打印。另外，需要注意换行符 `\n` 会引起 IO 的清理操作，若下面的语句 `printf("global=%d test=%d Parent,my PID is %d",global,test,getpid());` 加上 `\n`，则调用 `_exit(0)` 的结果和调用 `exit(0)` 的结果是一样的。

2. `vfork` 的特点

将上述代码的 `pid = fork();` 改为 `pid = vfork();` 编译后运行结果如下：

```
[root@localhost root]# ./test
the test content!
fork test!
global=23 test=1 Child,my PID is 2849
global=25 test=3 Parent,my PID is 2848
```

可以看出，`vfork` 与 `fork` 区别在于共享的资源不一样，`vfork` 调用后，子进程先对 `global` 和 `test` 加 1，父进程运行时，在其基础之上再加 2，得到上述运行结果。即 `vfork` 的特点是：在调用 `execv` 或者 `exit` 前子进程对变量的修改会影响到父进程，即他们是共享的；

特别注意：父进程等待子进程调用 `execv` 或 `exit` 才继续执行。则若子进程依赖父进程的进一步动作时，父进程又必须阻塞到子进程调用 `execv` 或者 `exit` 才会往下执行，此时就会造成“死锁”。读者可自己设计测试一下这种“死锁”状态。